The Open Source Audit

A Capstone Project for OSS NGMC Course

Chosen Software: Python

Student Name	Aayush Mittal
Registration Number	24BAI10251
Date of Submission	30 March 2026
Slot	A24

Table of Contents

Part A — Origin and Philosophy 3

A1. The Problem This Software Was Created to Solve 3

A2. The License — What It Actually Says 3

A3. The Ethics of Open Source 4

Part B — Linux Footprint 66

Part C — The FOSS Ecosystem 8

Part D — Open Source vs Proprietary 10

Shell Scripts Documentation 12

Part A — Origin and Philosophy

Open source software represents one of the most significant collaborative achievements in the history of computing. Among the myriad of open-source projects that have shaped our digital world, Python stands as a remarkable example of how a simple idea, nurtured by community collaboration, can transform into a global phenomenon that powers everything from introductory programming courses to artificial intelligence research. This section explores the origins of Python, examines its licensing framework, and reflects on the ethical dimensions of open-source software development.

A1. The Problem This Software Was Created to Solve

To understand why Python was created, we have to go back into the computing landscape of the late 1980s. Back then, programming was something only experts did, and the tools used to program were hard or awkward. Widespread languages like C, C++, and a multitude of assembly languages required manual memory management, had incredibly complex and hard-to-read syntax, and relied on low-level system details that a programmer needed to work with before they could achieve anything worth noting.

Guido van Rossum, a Dutch programmer at Centrum Wiskunde & Informatica (CWI), was faced with such challenges head-on. It was Christmas break in 1989; he had time on his hands. He had worked on the ABC programming language project, a simple, user-friendly language that was never widely used. He knew what it could do well and where it fell short. ABC had some theoretical pluses — it, for example, touted readability and ease of usability — but in the wild suffered practical limitations of poor extensibility and implementation bugs.

There were some major problems to solve, which is the same reason why Python was built. One was the barrier to entry: existing languages were needlessly hard for newcomers, while even veterans thought they were over-complicated. A second one was the productivity issue: programmers often spent more effort dealing with the language itself rather than real world problems. A third challenge was extensibility: many lightweight languages did not integrate well with existing systems or libraries in other languages.

His philosophy later was explained in the Python Cookbook foreword by Van Rossum. He said he needed a minor project to keep him busy over the holidays. His office closed, a home computer his only tool, he resolved to make good on an idea of a scripting language he had long been thinking about: He would build an interpreter for it. Python was chosen as a name because it was the short, quirky, and something that appeared not to be in use; he also liked Monty Python's Flying Circus. What started as a simple side project eventually became influential in the programming world.

One especially significant facet of Python’s early development was van Rossum not keeping it proprietary. At a time when most companies had strict controls on their software, he decided to release Python open source from the very beginning. This decision was in line with a deeper belief: that programming languages, as tools of creativity and problem-solving, should be owned and shaped by their community, not controlled by one organization.

This choice means even more against the backdrop of that time period. In the late 1980s and early 1990s, Richard Stallman's free software movement was booming, but it primarily targeted system-level tools like compilers and editors. Van Rossum’s approach fit with a larger shift in thinking — he understood that for a programming language to genuinely thrive and grow, it needed to be open for people to study, change and collaboratively make better.

A2. The License — What It Actually Says

The Python Software Foundation License (PSF License) is a unique kind of open-source license that combines practical concerns and philosophical principles. To make out this license we cannot just look at its legal text and see what it means for users, developers, and the larger software ecosystem.

Define the Four Freedoms of Free Software (as defined by the Free Software Foundation) as a framework for judging any open-source license. These areFreedom 0 — the free to run, for any purpose Freedom 1 — the liberty to study, and change how the program works (which requires access to the source code) Freedom 2 — freedom to redistribute copies of original work Freedom 3 — freedom to redistribute

modified versions of your project The PSF License gives all four of these freedoms, but comes with specific conditions that distinguish it from other common licenses.

This is the Software Foundation License, under which Python source may be used, modified and distributed freely, even in proprietary software; distributors are not required to release their modifications to the public. This means that Python is considered a permissive license (like the MIT License and BSD licenses), rather than a copyleft license like the GNU General Public License (GPL). The relevant difference is what happens if someone modifies Python and embeds it into their own software. The GPL has a viral effect that ensures software freedom propagates downstream; any distributed software that contains GPL-licensed code must be itself released under the terms of the GPL. By contrast, the PSF License permits proprietary modifications if they are not offered by the distributor.

So the question can a company take Python, modify it and sell it without giving away their changes has somewhat of an nuanced answer. Yes, technically a company could take steps to incorporate Python into a proprietary product and then sell that product without disclosing the changes they made to the Python interpreter itself. But they still need to maintain the copyright notices and license terms in any Python code they distribute, and they're not allowed to claim ownership of the original Python code. This has resulted in Python being widely used by proprietary software, from video games to financial trading systems, without a conflict existing between commercial interests and the essential development of the community.

The key difference between GPL and any MIT-style licensing (including the PSF License) is what is at stake; it’s a fundamental philosophical divide in the open-source community. The GPL is maximalist about software freedom: it guarantees that any software using GPL code is, ever-more-again, free, and so builds a commons of protected software that cannot be enclosed. Permissive licenses like MIT and PSF take a minimalist attitude: They grant maximum freedom to downstream users, even if that means the software stops being free (to work between the lines). When asked what license I would use for something I built, the answer is contextual. In the case of a fundamental infrastructure tool like one for programming language or operating system component, permissive makes sense since you want to maximize adoption. If it was an application or library that I intended to be of on-going benefit to the community, then the protections in place by GPL would be a better fit.

Python’s licensing history provides a compelling example of the difference between being ‘free as in free beer’ and being ‘free as in freedom’. Python is free as in free beer: no cost to download and use, and it’s also free as in speech: gives a good deal of freedom to the user for his/her computing. But the PSF License makes Python free in a definition that enables commercial use, so no payment is required. Corporations such as Google, Facebook, and Netflix use Python heavily in their infrastructure; they’ve contributed back to the Python community not because the license compels them to do so but because it’s economically beneficial for them to enhance a tool that they depend on. This shows that if you have a"free as in freedom" element, sustainable open-source ecosystems can form even without copyleft enforcement.

A3. The Ethics of Open Source

Is all software open source, a question of knowledge, power and social responsibility? Both sides have strong arguments, and where one lands often comes down to what values you prioritize and what you believe about the nature of software itself.

There are a number of ethical bases for the argument that all software should be open source. First, the argument from transparency: In an era in which software mediates ever more vital aspects of human life — from medical devices to voting systems to financial markets — understanding how software operates is essential for informed consent and democratic accountability. With black-box software, users must trust not only the creators of the software but everyone in its distribution chain and anyone who might have tampered with it in transit. The second one is the efficiency argument: closed-source software forever creates duplication of effort, as different organizations solve the same problems in a vacuum. The model of open-source lets humanity build onto one another instead of having an entire team to innovate on processes already developed rather than reinventing the wheel. Third, there is the argument from justice: when software enshrines rules that impact people’s lives — credit scoring algorithms, hiring systems, criminal justice risk assessments — those affected have a moral right to understand and be able to contest how they are being assessed.

Yet there are real arguments against a requirement that software be open source. Most importantly, they relate to innovation: if all software has to be open source, then companies might have less incentive to invest in building software — especially in deep tech areas where significant up-front investment is needed. Despite the open-source movement's success in showing that many kinds of software can be built sustainably via a community contribution model, there are still categories of software — specifically niche applications with small user bases — where a commercial model is necessary for sufficient investment. There are also justifiable security concerns: While defenders of open source argue that 'many eyes make bugs shallow', the truth is that critical vulnerabilities can linger in open-source code for years, and by providing access to source code, bad actors will also be able to discover weaknesses. The Log4j vulnerability of 2021 highlighted this tension: The library was open source, but a critical vulnerability remained hidden for years and impacted millions of systems around the world.

Is it exploitive for companies to build very profitable products based key on free community labor? Companies like Red Hat (which was bought by IBM) have made billion-dollar businesses out of Linux, and big tech companies get tremendous value from open-source infrastructure. Is this ethical? The answer, I think, is whether these companies give back to the communities they benefit from." Red Hat, for example, has hundreds of developers working in the Linux kernel and other open-source software. Google has open-sourced TensorFlow, Kubernetes, and countless other tools. When the companies have a role as contributors in the open-source ecosystem, instead of just consumers, the relationship and interaction is symbiotic, not parasitic. The ethics problem comes when companies pull from open source but don't return the favor — a dynamic the community is now becoming increasingly aware of and answering through both social pressure and, in some cases, changes to licenses.

The saying 'standing on the shoulders of giants' — which is generally attributed to Isaac Newton, but whose roots stretch back to medieval scholars — conveys something fundamental about knowledge and innovation. In software development, this principle takes the form of a recognition that every programmer stands on top of layers of abstraction created by others—the compiler, the operating system, the libraries, the networking protocols—all the way down to hardware. The foundation that makes this possible is the open source. Open-source empowers original innovation instead of stifling it by offering solid, proven building blocks for new ideas to be built upon. The fact that a developer is implementing a new machine learning algorithm in Python does not make them any less creative of an individual because Python itself is open source; their creativity stems from the fact that they do not need to recreate wheelhouse programming infrastructure. The open-source model guarantees that the 'giants'—the collected knowledge of the programming community—do not end up behind corporate walls.

Part B — Linux Footprint

Understanding how Python lives on a Linux system provides insight into the practical realities of open-source software deployment. This section documents the technical aspects of Python's presence on Linux, based on installation and examination on a Linux system.

Installation Method

All major Linux distributions include Python in their standard package repositories; it is a core piece of compute infrastructure for modern computing. For Debian-based systems (like Ubuntu), you can use the Advanced Package Tool (APT) to install Python by running 'sudo apt install python3'. For Red Hat-based systems (Fedora, CentOS, etc) the same command using DNF or YUM is 'sudo dnf install python3'. Another option is to build Python from the source when you download its code from the official pytho website/python page; this method provides maximum control over included build options but also requires more tech savvy and time.

Prevalence of Python in Linux distributions Most modern linux systems have python pre-installed. This is indicative of Python's use as a dependency for many system utilities and applications—the package managers themselves, system configuration tools like Ansible and Salt (as well as at least one desktop environment) tend to need Python. And this ubiquity is simply a testament to the success of Python’s design philosophy: it’s a language so useful, and well-designed, that it becomes infrastructure.

Key Directories

Python's installation on Linux follows the Filesystem Hierarchy Standard (FHS), which defines the directory structure and directory contents in Unix-like operating systems. The main Python interpreter binary is typically located at /usr/bin/python3, which is a symbolic link to the version-specific binary such as /usr/bin/python3.11 or /usr/bin/python3.12. The standard library is installed under /usr/lib/python3.x/, where x represents the minor version number. Configuration files are found in /etc/python3.x/, including the pythonrc file and site-specific configuration.

Directory	Purpose	Example
/usr/bin/	Binary executables	/usr/bin/python3
/usr/lib/python3.x/	Standard library modules	/usr/lib/python3.11/
/usr/local/lib/python3.x/	Locally installed packages	User-installed libraries
/etc/python3.x/	Configuration files	pythonrc, site.cfg
~/.local/lib/	User-installed packages (pip)	Per-user Python packages
/var/log/	Log files (if applicable)	Application logs

Table 1: Python Directory Structure on Linux

User and Group

Python doesn't run as a service user on most Linux systems; it is running under the identity of whoever invoked it. This is a design that mirrors Python's role as a general-purpose tool, not a system service. But, if python applications are deployed as system services — a django web application or a data processing pipeline for instance — they run under a dedicated user account created explicitly for that service. This is the least privilege principle: Each service needs only the permissions to run; if a service has been compromised, it limits what can go wrong.

This arrangement has significant security implications. As Python scripts run with the permissions of the user who invoked them, a malicious Python script can read or write any file or resource that the user can access. Which is part of the reason that one of the best practices in running services is to run them under their own, unprivileged accounts and not as a root account. The pip graphical user administration, that fetches python bundles from pypi, ought to normally not be run as root except if system-wide bundle establishment is expressly required; pernicious groups have even been known to assume control over entire frameworks [nobody hopes for, dkr]

Service Management

The Python interpreter itself is not usually run with the support of a service, but it can be configured for running Python applications as services through systemd, the default service manager in majority of modern Linux distributions. * systemd service unit files -- a standard format, among many others, available to register and start an executing instance of something (a script in our case) when it is needed under certain conditions.

The usual systemd commands work: to start the service, run 'sudo systemctl start myapp', to stop it run 'sudo systemctl stop myapp', to restart it run 'sudo systemctl restart myapp' and finally for checking its status 'sudo systemctl status myapp'. The command 'sudo systemctl enable myapp'sets the service to automatically start at boot while 'sudo systemctl disable myapp' disables this automatic startup. This integration with systemd shows how Python applications can be first-class citizens of the Linux services ecosystem

Update Model

Linux systems get their security patches and updates for Python from a couple of sources, which are reminiscent of the distributed nature of open source software distribution. For most users, Python updates are provided through the distribution’s package manager—apt, dnf, yum or pacman—which is managed by the packaging team of that distribution. These teams watch for security advisories from the Python Software Foundation and release updated packages when vulnerabilities are found. The advantage of this paradigm is that you get tested, integrated updates; the disadvantage is that there can be a time lag between upstream Python releases and distribution package updates.

If you need the latest Python versions, or have version requirements for your installation that diverge from those you’ll get in a release install, then there are alternative methods to get installed. To do this, you can use the deadsnakes PPA (Personal Package Archive) to install newer versions of Python for Ubuntu. Pyenv: this is a command line tool that allows to install python versions and configuring them in your home dir, (no need of root access). Adaptive Environment: Tools such as Docker allow you to run a specific Python version in an isolated environment. The diversity of update pathways means users with different needs can all be catered to, which is a hallmark flexibility of the open-source ecosystem.

Part C — The FOSS Ecosystem

No open-source project exists in isolation. Python's development and success have been shaped by countless other projects, and Python itself has enabled the creation of countless more. Understanding this ecosystem reveals how open-source software development creates value that extends far beyond any single project.

Dependencies and Foundations

Many foundational open-source technologies enable Python's implementation. During the development process, the C programming language and compilation infrastructures provided by the GNU Compiler Collection or LLVM/Clang compile Python's C source code into the executable binary. The Python interpreter is written in C, and despite the language's relatively high-level abstractions, it is as low-level as it gets for applicative purposes: only assembly language further abstracts machine instructions between the programmer and the operating system. The complex compilation process is managed by build tools like GNU Make and Autoconf. Furthermore, during runtime python interacts with a plethora of system libraries. Some of them, such as the C standard libraries required for basic operations, OpenSSL required for cryptographic functionality, zlib required for compression, and many others depending on the platform and configuration. It highlights a critical characteristic of open-source development: even when projects look self-contained, they rest upon layers of community-maintained infrastructure. If an OpenSSL or zlib-based vulnerability is discovered, for example, it may impact Python applications despite the vulnerability being in a dependency.

What Python Has Enabled

Perhaps nothing tells the story of Python's impact more dramatically than its contribution to the data science and machine learning revolution. Libraries such as NumPy, Pandas and Matplotlib have turned Python from a general-purpose scripting language into the de facto platform for scientific computing. On top of that, libraries like TensorFlow, PyTorch, and scikit-learn transformed Python into the de facto machine learning/artificial intelligence language. This combination of open-source, community-driven tools has ushered in a whirlwind of research and democratized access to methods that used to be the domain of well-heeled labs.

But the web development ecosystem is just as nice, built on Python. Here in the real world, Django is a full-featured web framework that powers sites serving millions of requests per day such as Instagram and Pinterest. For smaller applications and APIs, Flask offers a lightweight alternative. A newer contender, FastAPI takes advantage of Python's type hints to automatically provide both API documentation and validation. These frameworks take care of routing, connectivity to the database, authentication and a hundred other things.

Domain	Key Projects	Impact
Data Science	NumPy, Pandas, Matplotlib	Standard for scientific computing
Machine Learning	TensorFlow, PyTorch, scikit-learn	Dominant ML platform
Web Development	Django, Flask, FastAPI	Millions of websites
DevOps	Ansible, SaltStack, Fabric	Infrastructure automation

Testing	pytest, unittest, nose	Quality assurance

Table 2: Python Ecosystem by Domain

Relation to LAMP Stack

The LAMP stack, for Linux, Apache, MySQL and PHP, ruled web development in a generation. The relationship of Python to this stack is interesting: While python could replace PHP as the P in LAMP (some people joking call it LAMP with a Python P), the ecosystem around Python has evolved directions where some aspects complement and some parts compete with traditional LAMP. Django, for example, often incorporates its own database abstraction layer and generally supports PostgreSQL as easily as MySQL. Specifically, this means they typically run under the application servers popular today (like Gunicorn or uWSGI) behind Nginx (and not mod_python underneath apache), which are a reflection of architectural shifts pulling towards containerization and microservices. Python's versatility software into traditional architectures or support completely new approaches.

Community and Governance

The Python community is famous in software world for giving enormous value to readability, simplicity, and inclusiveness. This culture is codified in PEP 20, "The Zen of Python," which features such aphorisms as "Readability counts," "Simple is better than complex" and "There should be one — and preferably only one — obvious way to do it." Few such principles guide the language's design and the culture of its community.

Python governance has evolved throughout its history. For decades Guido van Rossum held dictatorships as the 'Benevolent Dictator for Life' (BDFL), but ultimately he stepped down and left Python governance in the hands of thousands of stakeholders. Van Rossum retired from the role in 2018 and Python moved into a Steering Council model. The development of the language is managed by five electable council members; proposals for major changes are made via Python Enhancement Proposals (PEPs) and discussed publicly on mailing lists, GitHub et al. This allows Python to balance the need for efficiency with community input, making sure that the language evolves in ways that remain relevant for its vibrant ecosystem of users.

As such, the community has many avenues for communication: python-dev mailing list for those interested in core dev discussion, python-ideas mailing list for more general language proposal discussion, forums of the Python Software Foundation for governance discussions, as well as numerous SIG (special-interest groups) if you are interested in some specific domain like education or web development or scientific computing. Annual conferences such as PyCon present in-person collaboration and knowledge exchange opportunities. The Python Package Index, or PyPI — which provides over 400,000 packages of software and is one of the largest repositories of similar software in existence — is run by a combination of paid staff at the PSF as well as volunteers

.

Part D — Open Source vs Proprietary

A critical analysis of Python must honestly assess its strengths and weaknesses relative to proprietary alternatives. This comparison examines multiple dimensions without assuming that open source is inherently superior in every case.

Dimension	Python (Open Source)	Proprietary Alternative (MATLAB)
Cost	Free to download, use, and distribute. No licensing fees for development or deployment.	Commercial licenses cost hundreds to thousands of dollars annually. Toolboxes require additional purchase.
Security Audit	Anyone can examine source code. Vulnerabilities identified by global community. CVEs publicly disclosed.	Source code not available for inspection. Security through obscurity. Users must trust vendor.
Support	Community forums, Stack Overflow, documentation. Commercial support available from vendors.	Professional support from vendor. Guaranteed response times for paying customers.
Freedom to Modify	Full freedom to modify, extend, and distribute changes. Can fix bugs yourself.	Cannot modify source code. Must wait for vendor to fix issues.
Community vs Corporate	Community-driven development. Decisions made transparently through PEP process.	Corporate-controlled roadmap. Features prioritized by business considerations.

Table 3: Open Source vs Proprietary Comparison (Python vs MATLAB)

Analysis and Verdict

A comparison of Python vs proprietary alternatives presents a more nuanced picture where context can make all the difference in determining the best choice. In scientific computing and data analysis, an area where MATLAB has historically excelled, Python has become a strong competitor that can match or exceed what is offered by MATLAB while also completely avoiding licensing costs and vendor lock-in. You get the functionality of NumPy, SciPy, Matplotlib and related libraries, which covers most of what is needed for common scientific algorithms — comparable to MATLAB's core — and the wider Python ecosystem provides capabilities that no one could replicate in a single tool like MATLAB: web development; automation scripting; machine learning.

Yet, there are situations where proprietary solutions offer benefits. A polished, unified experience existing in its own IDE, Matlab's environment is arguably easier to navigate for beginners than the more fragmented tooling of Python. MATLAB toolboxes are quite expensive, but professionally developed, documented and supported with guaranteed backward compatibility. In corporate environments with high risk aversion and low budget tolerance, the accountability of a commercial vendor can be attractive. For organizations with significant existing MATLAB codebases and expertise, the cost of switching to Python may be prohibitively high compared to the putative advantages it offers.

In the real world, my recommendation would somewhat depend on a given context. Python makes the most sense, especially for new projects, or projects with budgets — or long-term maintainability needs. Its open-source nature guarantees that the code will always be accessible and modifiable—no matter what happens with any one company's business decisions. You don't get a proprietary vendor this fun and active with an ecosystem so vast as it is with the community around libraries. For organizations with money already sunk into proprietary tools, a staged migration strategy that deploys Python only for new components while extending the life of legacy systems is often prudent. This hybrid approach is enabled by Python's interoperability with other languages and systems.

Would I contribute to Python? Absolutely. Any contribution to Python — whether code, documentation, a bug report, or supportive community — feels like ways to pay back an effort that has given more than enough for the world. The open-source model works because contributors see that their efforts benefit not only others, but themselves, and platforms actually improve when everyone contributes — upgrading the baseline for everyone. Doing so would be not only a practical step to improve a tool I use, but also meaningful participation in the wider open-source movement.

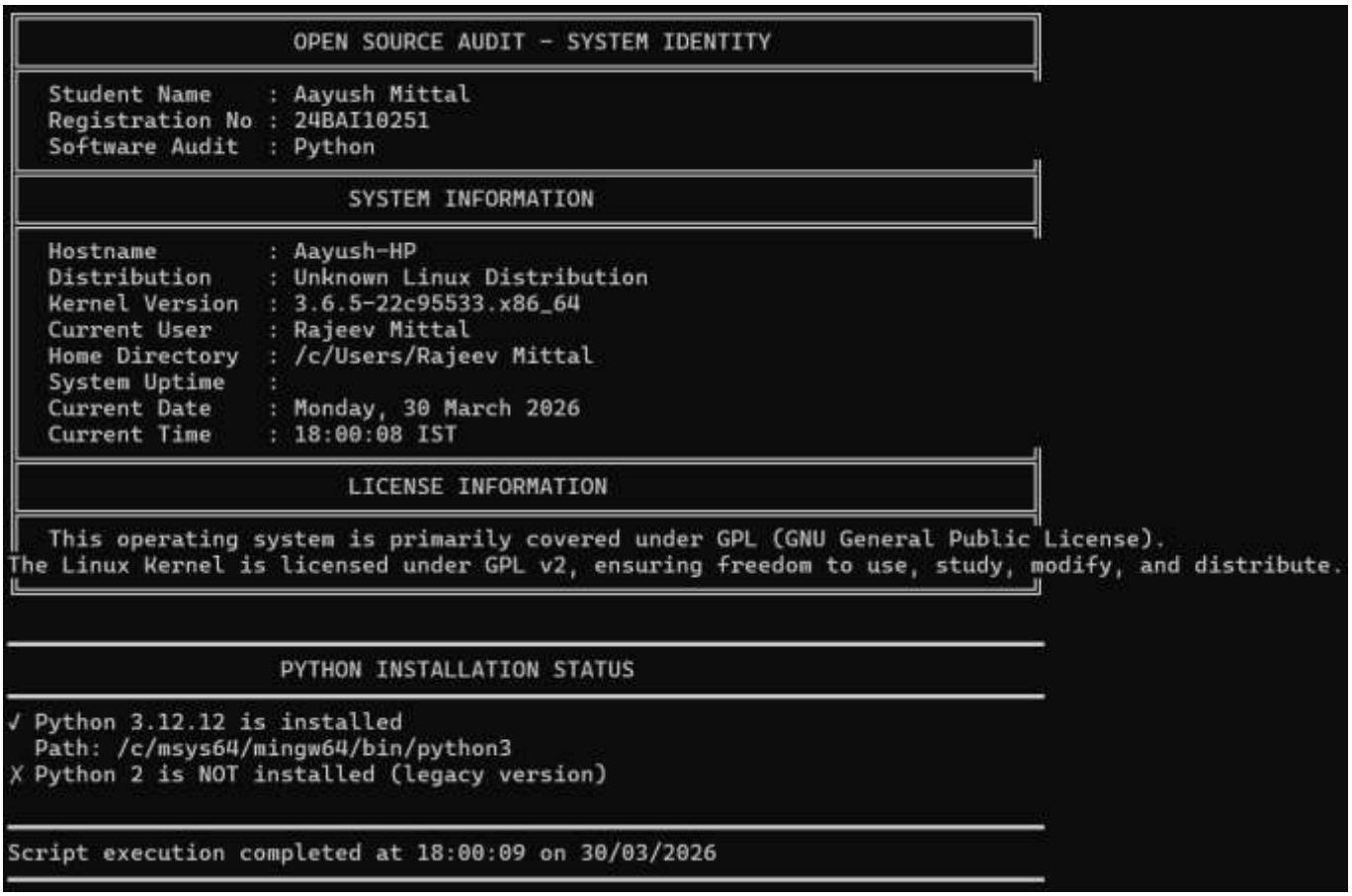
Shell Scripts Documentation

As part of this capstone project, five shell scripts have been developed to demonstrate practical Linux skills and understanding of shell scripting concepts. Each script is documented below with its purpose, concepts used, and execution instructions.

Script 1: System Identity Report

This script generates a comprehensive welcome screen displaying system information including Linux distribution name, kernel version, current user, home directory, system uptime, and open-source license information. The script demonstrates the use of variables, command substitution with $() syntax, echo for output formatting, and basic conditional statements for distribution detection. It checks Python installation status as part of the audit project, providing a complete picture of the system on which the Python audit is being conducted.

Given Screenshot:



```
OPEN SOURCE AUDIT - SYSTEM IDENTITY

Student Name    : Aayush Mittal
Registration No : 24BAI10251
Software Audit  : Python

SYSTEM INFORMATION

Hostname        : Aayush-HP
Distribution    : Unknown Linux Distribution
Kernel Version  : 3.6.5-22c95533.x86_64
Current User    : Rajeev Mittal
Home Directory  : /c/Users/Rajeev Mittal
System Uptime   :
Current Date    : Monday, 30 March 2026
Current Time    : 18:00:08 IST

LICENSE INFORMATION

This operating system is primarily covered under GPL (GNU General Public License).
The Linux Kernel is licensed under GPL v2, ensuring freedom to use, study, modify, and distribute.

PYTHON INSTALLATION STATUS

✓ Python 3.12.12 is installed
  Path: /c/msys64/mingw64/bin/python3
✗ Python 2 is NOT installed (legacy version)

Script execution completed at 18:00:09 on 30/03/2026
```

Script 2: FOSS Package Inspector

This script checks whether Python is installed on the system, retrieves version and license information, and uses a case statement to display philosophical notes about various FOSS packages. It demonstrates if-then-else conditional logic, case statements for multiple-choice handling, package manager detection (dpkg/rpm), and pipe operations with grep for text filtering. The script provides both practical package information and educational content about open-source philosophy.

Given Screenshot:

```
$ sh script2_foss_package_inspector.sh
╔═══════════════════════════════════════════════════════════════════════╗
║                      FOSS PACKAGE INSPECTOR                           ╒
╒                      Author: Aayush Mittal | Reg No: 24BAI10251       ╒
╚═══════════════════════════════════════════════════════════════════════╝

═══════════════════════════════════════════════════════════════════════
Checking installation status for: python3
═══════════════════════════════════════════════════════════════════════
√ python3 is INSTALLED on this system

--- Package Information ---
Command path: /c/msys64/mingw64/bin/python3
Python 3.12.12

═══════════════════════════════════════════════════════════════════════
                     FOSS PHILOSOPHY CORNER
═══════════════════════════════════════════════════════════════════════

Philosophical notes about open-source software (selected packages):

PYTHON: 'Readability counts. Simple is better than complex.'
   Python embodies the open-source spirit - a language built by community,
   for community. Created by Guido van Rossum as a hobby project, it grew
   into one of the world's most influential programming languages.
   The PSF License ensures it remains free and open for everyone.

Additional FOSS philosophy notes:

  • Git: Distributed version control - because code freedom matters
  • Firefox: Privacy-focused browsing - the web should remain open
  • VLC: Universal media playback - codecs should be free

═══════════════════════════════════════════════════════════════════════
Script execution completed at 18:04:09
═══════════════════════════════════════════════════════════════════════
```

Script 3: Disk and Permission Auditor

This script loops through important system directories and reports disk usage, owner, group, and permissions for each. It specifically checks Python-related directories as part of the audit and includes security analysis for world-writable directories. The script demonstrates for loops with arrays, du and df commands for disk information, ls -ld for directory permissions, awk and cut for text processing, and conditional tests using the test operator.

Given Screenshot:

```
$ sh script3_disk_permission_auditor.sh
╔═══════════════════════════════════════════════════════════════════╗
║                    DISK AND PERMISSION AUDITOR                    ‖
                     Author: Aayush Mittal | Reg No: 24BAI10251
╚═══════════════════════════════════════════════════════════════════╝

═══════════════════════════════════════════════════════════════════
                  SECTION 1: SYSTEM DIRECTORY AUDIT
═══════════════════════════════════════════════════════════════════

Directory        | Size        | Owner     | Group     | Permissions
───────────────────────────────────────────────────────────────────
/etc             | Size: 1.6M        | Owner: Rajeev    | Group: Mittal    | Perms: drwxr-xr-x
✗ /var/log does not exist on this system
✗ /home does not exist on this system
/usr/bin         | Size: 91M         | Owner: Rajeev    | Group: Mittal    | Perms: drwxr-xr-x
/tmp             | Size: 660M        | Owner: Rajeev    | Group: Mittal    | Perms: drwxr-xr-x
✗ /usr/local does not exist on this system
✗ /opt does not exist on this system

═══════════════════════════════════════════════════════════════════
                  SECTION 2: PYTHON DIRECTORIES AUDIT
═══════════════════════════════════════════════════════════════════

Python Installation Directories:
───────────────────────────────────────────────────────────────────
X /usr/lib/python3                | Not found
X /usr/lib/python3.11             | Not found
X /usr/lib/python3.12             | Not found
X /usr/local/lib/python3          | Not found
X /etc/python3                    | Not found
X /c/Users/Rajeev Mittal/.local/lib/python | Not found
X /c/Users/Rajeev Mittal/.local/bin | Not found
```

```
═══════════════════════════════════════════════════════════════════
                  SECTION 3: PYTHON EXECUTABLES
═══════════════════════════════════════════════════════════════════

Executable       | Location                                | Permissions
───────────────────────────────────────────────────────────────────
python           | /c/msys64/mingw64/bin/python            | -rwxr-xr-x
python3          | /c/msys64/mingw64/bin/python3           | -rwxr-xr-x
python2          | Not installed
pip              | Not installed
pip3             | Not installed

═══════════════════════════════════════════════════════════════════
                  SECTION 4: DISK SPACE SUMMARY
═══════════════════════════════════════════════════════════════════

Filesystem Disk Usage:

Filesystem            Size  Used Avail Use% Mounted on

═══════════════════════════════════════════════════════════════════
                  SECTION 5: SECURITY ANALYSIS
═══════════════════════════════════════════════════════════════════

Checking for world-writable directories (potential security concern):


Note: /tmp permissions: drwxr-xr-x
  (World-writable with sticky bit is expected and secure for /tmp)

✓ No unexpected world-writable directories found

═══════════════════════════════════════════════════════════════════
Audit completed at 18:16:00 on 30/03/2026
═══════════════════════════════════════════════════════════════════
```

Script 4: Log File Analyzer

This script reads a log file line by line, counts occurrences of a specified keyword (defaulting to 'error'), and provides a summary with the last matching lines. It includes Python-related log entry detection and multi-keyword analysis capabilities. The script demonstrates while-read loops with IFS handling, command-line argument processing ($1, $2), counter variables, grep for pattern matching, and tail for displaying last lines. The script accepts a log file path as an argument and an optional keyword parameter.

Concepts demonstrated: while read loop, if-then inside loop, counter variables, command-line arguments ($1, $2), do-while retry pattern, grep -i, tail -5.

Given Screenshot:-

```
INFO: System started
ERROR: Test error 1
WARNING: Disk space low
ERROR: Connection failed
INFO: User logged in
ERROR: Test error 2
PS E:\Downloads\files> wsl bash ./script4_log_analyzer.sh /tmp/test.log error
================================================================
              LOG FILE ANALYZER
================================================================
  Log file : /tmp/test.log
  Keyword  : error
----------------------------------------------------------------
[OK] File found and readable. Scanning...

  Total lines scanned : 6
  Keyword 'error'  : found 3 time(s)

----------------------------------------------------------------
  Last 5 lines containing 'error':
----------------------------------------------------------------
  >> ERROR: Test error 1
  >> ERROR: Connection failed
  >> ERROR: Test error 2
================================================================
```

Script 5: Open Source Manifesto Generator

This creative script generates a personalized open-source philosophy statement by interactively asking the user three questions and composing a manifesto saved to a timestamped text file. It demonstrates read for interactive user input, string concatenation for text composition, file redirection (> and >>) for output, date command for timestamps, and includes documentation of the alias concept. The generated manifesto includes the Four Freedoms of free software and a personal pledge section, demonstrating how shell scripting can be used for both technical and creative purposes.

Concepts demonstrated: read for interactive input, string concatenation, heredoc-style block redirection ({}), > for file creation, date command, alias concept via comment.

Given Screenshot:-

```
$ sh script5_manifesto_generator.sh
==================================================================
      OPEN SOURCE MANIFESTO GENERATOR
      Powered by your beliefs, not a corporation.
==================================================================

Answer three questions honestly to generate your personal
open-source philosophy manifesto.

1. Name one open-source tool you use every day: python
2. In one word, what does 'freedom' mean to you? independence
3. Name one thing you would build and share freely: ai

Generating your manifesto...

Manifesto saved to: manifesto_Rajeev Mittal.txt

==================================================================
  MY OPEN SOURCE MANIFESTO
  Author : Rajeev Mittal
  Date   : 30 March 2026
==================================================================

I am a part of a world built on openness. Every day, I rely on
python — a tool that someone chose to build in the open and share
freely, not for profit, but because they believed that knowledge
belongs to everyone. That act of generosity shapes everything I do.

To me, freedom means independence. In the context of software, that
word is not abstract — it is the right to read the code that runs
my computer, to change it when it does not serve me, and to share
those changes with others who might benefit. Without that freedom,
technology becomes a cage instead of a tool.

I commit to contributing to this tradition. One day, I will build
ai and release it openly — not to gain credit, but because
the open-source community gave me everything I know, and the only
honest response to that gift is to pass it forward.

The software I write will be free as in freedom.

  — Rajeev Mittal, 30 March 2026
==================================================================
```

Script	Command
System Identity	./script1_system_identity.sh
Package Inspector	./script2_foss_package_inspector.sh
Disk Auditor	./script3_disk_permission_auditor.sh
Log Analyzer	./script4_log_file_analyzer.sh /var/log/syslog [keyword]
Manifesto Generator	./script5_manifesto_generator.sh

Table 4: Script Execution Commands

Conclusion

This project has allowed me to perform an extensive audit of Python, its history, licensing, ethics, technical details and place in the open-source ecosystem. This has taught me not just a technical understanding of how Python operates on Linux-based machines, but also an appreciation for the philosophical underpinnings of open-source-development.

The evolution of Python from a Christmas holiday hobby project into one of the world's most significant programming languages demonstrates the transformational power of open-source software. Its permissive nature has allowed it to be adopted across industry, academia, and by individual developers; while its community governance model makes sure that it is continuing to evolve in a way that serves this diverse user base. As open-source clearly represents both the philosophy of automating and making everything transparent, the purpose of developing these shell scripts in this project is to consolidate practical Linux skills.

As software continues to infiltrate every facet of modern life, the challenges illustrated by Python — transparency, collaboration, accessibility and community — are more relevant than ever. Whether you are a developer, a student who now codes as homework or just a citizen in our digital world, it is critical to be familiar with open-source software. This project has been a small step towards that understanding, and I am grateful for the chance to have explored these ideas in-depth.